

# DANKE MOTORSPORT

## Análise Técnica & Roadmap de Implementação

Avaliação na perspectiva de Engenheiro de Software Sênior e Professor Universitário  
Stack: React · C# .NET 10 · Entity Framework Core · PostgreSQL (Supabase)  
Data: Junho de 2026 | Versão do repositório analisado: branch main

| Critério de Avaliação                | Peso          | Status Atual |
|--------------------------------------|---------------|--------------|
| 1. Acesso apenas após autenticação   | 0,5 pt        | ■ AUSENTE    |
| 2. Cadastro de novos usuários        | 0,5 pt        | ■■ PARCIAL   |
| 3. Atualização de cadastro           | 0,5 pt        | ■ AUSENTE    |
| 4. Base de dados no back-end         | 0,5 pt        | ■ PRESENTE   |
| 5. Compartilhar dados entre usuários | 0,75 pt       | ■ AUSENTE    |
| 6. Front-end responsivo              | 1,0 pt        | ■ AUSENTE    |
| 7. Deploy 24/7 em servidor           | 0,75 pt       | ■ AUSENTE    |
| 8. Organização da entrega/código     | 1,0 pt        | ■■ PARCIAL   |
| 9. Documentação do código            | 0,5 pt        | ■ AUSENTE    |
| TOTAL EM RISCO                       | 5,5 / 6,0 pts | Agir já      |

# 1. Estado Atual do Repositório

A análise foi feita sobre o estado atual do branch **main**. O projeto possui uma estrutura de pastas coerente com a separação entre **frontend/** e **backend/**, porém a distância entre o planejamento documentado na pré-análise e o código efetivamente escrito é muito grande. A seguir, o que foi encontrado em cada camada.

## 1.1 Backend — C# .NET 10

- **AppDbContext.cs**: Correto. Três DbSet (Clientes, Funcionarios, Revisoes) mapeados via EF Core.
- **Migrations**: Existem e estão funcionais — sinal de que a conexão com o Supabase foi testada.
- **ClientesController.cs**: Apenas GET e POST sem autenticação, sem senha, sem hash.
- **FuncionariosController.cs**: Idêntico ao anterior — sem proteção alguma.
- **RevisaoController.cs**: Arquivo existe mas está **completamente vazio** (1 linha).
- **AuthController.cs**: Citado na pré-análise, **não existe** no repositório.
- **Program.cs**: Sem JWT, sem CORS, sem UseAuthentication / UseAuthorization.

■■ ALERTA DE SEGURANÇA CRÍTICO: A senha do banco de dados está em texto puro no arquivo appsettings.json e esse arquivo foi commitado no git. Qualquer pessoa com acesso ao repositório tem credenciais de produção. Rotacione a senha no Supabase imediatamente e adicione appsettings.json ao .gitignore.

## 1.2 Frontend — React

- **App.jsx**: Retorna apenas um `<div className='container'>` vazio. O router não está conectado.
- **routes/index.jsx**: Define 2 rotas (/ e /auth) mas o roteador não é usado em main.jsx.
- **Auth.jsx**: UI bem feita (toggle login/cadastro, ícones, show/hide senha) mas **sem nenhuma chamada à API** — os formulários não submetem dados.
- **package.json**: react-icons e axios **não estão listados** como dependências, mas são importados em Auth.jsx. O projeto quebraria em produção.
- **CSS**: Nenhum @media query encontrado. A classe .inpt-form usa width: 25vw — inutilizável em mobile.
- **AuthContext.jsx, ClientDashboard.jsx, EmployeeDashboard.jsx**: Citados na pré-análise, não existem no repositório.

## 1.3 README

- O README descreve MongoDB como banco de dados, mas o projeto usa PostgreSQL. Isso é uma inconsistência que o avaliador notará.

# 2. Gap Analysis — Critérios vs. Código Real

| # | Critério (Peso) | O que falta |
|---|-----------------|-------------|
|---|-----------------|-------------|

|   |                                          |                                                                                                                                                                                         |
|---|------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Acesso só após autenticação (0,5)        | AuthController inexistente. JWT não configurado no Program.cs. Rotas protegidas e PrivateRoute no React inexistentes. Todos os endpoints são públicos hoje.                             |
| 2 | Cadastro de novos usuários (0,5)         | Frontend: formulário existe mas não faz POST. Backend: endpoint POST /danke/clientes existe mas o modelo Cliente não tem campo Senha. Sem hash BCrypt.                                  |
| 3 | Atualização de cadastro (0,5)            | Nenhum endpoint PUT/PATCH existe. Nenhuma página de perfil existe no frontend.                                                                                                          |
| 4 | Base de dados no back-end (0,5)          | ■ COBERTO. EF Core + PostgreSQL + Supabase configurados e migração executada.                                                                                                           |
| 5 | Compartilhar dados entre usuários (0,75) | RevisaoController vazio. Sem endpoint GET para funcionários verem revisões de clientes. Sem endpoint PATCH para atualizar status. Sem Kanban.                                           |
| 6 | Front-end responsivo (1,0)               | Nenhum @media query nos arquivos CSS. Inputs usam 25vw fixo (ilegível em mobile). Navbar sem menu hambúrguer.                                                                           |
| 7 | Deploy 24/7 em servidor (0,75)           | Sem evidência de deploy do frontend (ex: Vercel/Netlify) nem do backend (ex: Railway/Render/Azure). Supabase cobre o DB, mas a API precisa estar hospedada.                             |
| 8 | Organização código/entrega (1,0)         | ■■ PARCIAL. Estrutura de pastas ok. Mas: README diverge da stack, arquivo RevisaoController.cs vazio commitado, credenciais expostas, inconsistências de naming (id vs id_funcionario). |
| 9 | Documentação do código (0,5)             | Sem XML doc comments no C# (///). Sem JSDoc no React. Os comentários existentes são didáticos (ex: 'O EF faz o SELECT sozinho'), não de documentação profissional.                      |

## 3. Feedback de Código — Boas Práticas de Mercado

### 3.1 C# / .NET — O que está bom

■ Uso correto de Injeção de Dependência (DI) via construtor nos controllers. ■ Atributos [Required], [Key], [Column], [Table] e [ForeignKey] usados corretamente nos Models. ■ Separação de responsabilidades em Controllers/, Models/, Data/ — aderente ao padrão MVC. ■ Swagger habilitado para facilitar testes da API.

### 3.2 C# / .NET — O que melhorar

- **Sem async/await:** Todos os métodos de controller são síncronos. Em .NET, banco de dados sempre deve ser acessado com await e métodos \*Async do EF Core (ToListAsync, SaveChangesAsync). Isso bloqueia threads do servidor sob carga.
- **Sem DTOs (Data Transfer Objects):** Os controllers retornam os Models diretamente. Isso expõe o schema do banco, inclui dados sensíveis e cria acoplamento. Crie DTOs separados para request/response.
- **Sem validação de entrada:** POST /danke/clientes aceita qualquer payload. Adicione FluentValidation ou Data Annotations robustos.
- **Inconsistência de naming nos Models:** Cliente usa Id (int), Funcionario usa IdFuncionario (int). Padronize para Id em todos os Models e deixe o [Column] fazer o mapeamento para o banco.

- **TipoFuncionario e Cargo como int:** Use enum com [EnumMember] ou uma tabela de lookup. Valores mágicos (1, 2, 3) são impossíveis de manter.
- **DatFinalizacao como Required:** Uma revisão recém-criada não tem data de finalização. Esse campo deveria ser nullable (DateTime?).

### Exemplo de refatoração — Controller async com DTO:

```
// ANTES (ruim)
public IActionResult ListarClientes() {
    var c = _context.Clientes.ToList();
    return Ok(c);
}

// DEPOIS (mercado)
public async Task<IActionResult> ListarClientes() {
    var c = await _context.Clientes
        .Select(x => new ClienteResponseDto {
            Id = x.Id, Nome = x.Nome, Email = x.Email
        })
        .ToListAsync();
    return Ok(c);
}
```

## 3.3 React — O que está bom

■ Uso de useState para controle de estado do formulário (handleChange genérico bem feito). ■ Toggle login/cadastro com um único componente Auth.jsx — boa coesão. ■ Uso de react-router-dom para SPA routing. ■ CSS customizado com variáveis CSS (--accent-gold, --bg-dark) — correto e escalável. ■ Design visual premium coerente com a identidade da marca.

## 3.4 React — O que melhorar

- **App.jsx vazio + router desconectado:** O RouterProvider não está em main.jsx. A aplicação não roteia nada. Corrija: import { RouterProvider } from 'react-router-dom'; <RouterProvider router={router} />
- **Dependências faltando no package.json:** react-icons e axios são importados mas não declarados. Adicione: npm install react-icons axios
- **<a> dentro de <button> (Navbar):** HTML inválido. Use <Link> do react-router-dom no lugar de <a href> para navegação interna.
- **Formulários sem onSubmit:** Os <form> em Auth.jsx não têm handler de submissão. Sem preventDefault(), o formulário recarrega a página.
- **Sem feedback visual:** Não há loading state, mensagens de erro ou sucesso para o usuário.

## 4. Roadmap de Execução Priorizado

Abaixo está o roadmap ordenado por **impacto na nota × tempo de implementação**. As tarefas que desbloqueiam outras estão marcadas com [BLOQUEANTE]. A estimativa assume que você programa 2–3h por sessão.

### FASE 1 — Fundação (Sessão 1–2) · Desbloqueante para tudo

#### Tarefa 1.1 — Corrigir o roteamento do React [15 min]

Sem isso, nenhuma página funciona. Edite `frontend/src/main.jsx`:

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import { RouterProvider } from 'react-router-dom';
import router from './routes';
import './index.css';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <RouterProvider router={router} />
  </React.StrictMode>
);
```

#### Tarefa 1.2 — Instalar dependências faltando [5 min]

```
npm install react-icons axios
```

#### Tarefa 1.3 — Remover credenciais do repositório [10 min]

1. Adicione ao `.gitignore`: `appsettings.json` e `appsettings.Development.json`
2. Crie `appsettings.Example.json` com placeholders para documentar a estrutura.
3. Use variáveis de ambiente no servidor de deploy (nunca em arquivo commitado).
4. Rotacione a senha do Supabase imediatamente no painel deles.

### FASE 2 — Autenticação [Peso: 0,5 + 0,5 = 1,0 pt] (Sessão 2–4)

#### Tarefa 2.1 — Adicionar campo Senha ao Model + Migration [20 min]

Em `Models/Cliente.cs`, adicione:

```
[Required]
[Column("senha_hash")]
public string SenhaHash { get; set; } = string.Empty;
```

Depois rode: `dotnet ef migrations add AddSenhaCliente` e `dotnet ef database update`

#### Tarefa 2.2 — Instalar BCrypt e configurar JWT no Program.cs [30 min]

```
dotnet add package BCrypt.Net-Next
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
```

Em **Program.cs**, adicione (antes do `builder.Build()`):

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options => {
        options.TokenValidationParameters = new TokenValidationParameters {
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"]!)),
            ValidateIssuer = false,
            ValidateAudience = false
        };
    });

// E depois do Build():
app.UseAuthentication();
app.UseAuthorization();
```

### Tarefa 2.3 — Criar **AuthController.cs** [45 min]

Crie **backend/Controllers/AuthController.cs** com dois endpoints:

- **POST /danke/auth/registro**: Recebe nome, email, senha. Faz `BCrypt.HashPassword()`. Salva no banco. Retorna 201.
- **POST /danke/auth/login**: Busca cliente por email. Verifica `BCrypt.Verify()`. Gera e retorna JWT com claims (Clientid, Email, Role).

```
// Exemplo de geração de token
var tokenHandler = new JwtSecurityTokenHandler();
var key = Encoding.UTF8.GetBytes(_config["Jwt:Key"]!);
var tokenDescriptor = new SecurityTokenDescriptor {
    Subject = new ClaimsIdentity(new[] {
        new Claim(ClaimTypes.NameIdentifier, cliente.Id.ToString()),
        new Claim(ClaimTypes.Email, cliente.Email),
        new Claim(ClaimTypes.Role, "cliente")
    }),
    Expires = DateTime.UtcNow.AddHours(8),
    SigningCredentials = new SigningCredentials(
        new SymmetricSecurityKey(key), SecurityAlgorithms.HmacSha256Signature)
};
var token = tokenHandler.CreateToken(tokenDescriptor);
return Ok(new { token = tokenHandler.WriteToken(token) });
```

### Tarefa 2.4 — Proteger endpoints com **[Authorize]** [10 min]

Adicione **[Authorize]** no topo de `CientesController` e `FuncionariosController`. Mantenha `AuthController` sem o atributo (endpoints públicos para login/registro).

### Tarefa 2.5 — **AuthContext** + **PrivateRoute** no React [60 min]

Crie **src/context/AuthContext.jsx** com `useState` para o token JWT (armazenado em `localStorage`). Exponha `login()`, `logout()`, e `isAuthenticated`.

Crie **src/components/PrivateRoute.jsx**:

```
import { Navigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
```

```
export default function PrivateRoute({ children }) {  
  const { isAuthenticated } = useAuth();  
  return isAuthenticated ? children : <Navigate to='/auth' />;  
}
```

Em **Auth.jsx**, adicione o onSubmit que faz POST para /danke/auth/login com axios e chama login() do contexto.

## FASE 3 — Atualização de Cadastro [Peso: 0,5 pt] (Sessão 4–5)

---

### Tarefa 3.1 — Endpoint PUT /danke/clientes/{id} no backend [20 min]

Em ClientesController, adicione:

```
[HttpPut("{id}")]
[Authorize]
public async Task<IActionResult> AtualizarCliente(int id, [FromBody] ClienteUpdateDto dto)
{
    // Valide que o id do token == id da rota (segurança)
    var clienteId = int.Parse(User.FindFirst(ClaimTypes.NameIdentifier)!.Value);
    if (clienteId != id) return Forbid();

    var cliente = await _context.Clientes.FindAsync(id);
    if (cliente is null) return NotFound();

    cliente.Nome = dto.Nome;
    cliente.Telefone = dto.Telefone;
    await _context.SaveChangesAsync();
    return Ok(cliente);
}
```

### Tarefa 3.2 — Página de Perfil no React [30 min]

Crie **src/pages/Perfil.jsx** com formulário pré-preenchido (GET /danke/clientes/{id}) e botão salvar (PUT). Adicione a rota /perfil protegida com PrivateRoute.

## FASE 4 — Compartilhamento de Dados / Kanban [Peso: 0,75 pt] (Sessão 5–7)

---

### Tarefa 4.1 — Implementar RevisaoController.cs [60 min]

- **POST /danke/revisao** [Authorize(Roles="cliente")]: Cliente cria solicitação. IdCliente extraído do JWT claim, não do body.
- **GET /danke/revisao** [Authorize(Roles="funcionario")]: Lista todas as revisões com dados do cliente.
- **PATCH /danke/revisao/{id}/status** [Authorize(Roles="funcionario")]: Atualiza StatusRevisao e atribui IdFuncionario.

### Tarefa 4.2 — Dashboard do Cliente + Kanban do Funcionário no React [90 min]

- **src/pages/ClientDashboard.jsx**: Formulário de agendamento + lista das próprias revisões.
- **src/pages/EmployeeDashboard.jsx**: Quadro Kanban com colunas (Pendente / Em Andamento / Concluído). Cada card é uma revisão arrastável (use CSS simples com botões de avançar status).
- Ambas as rotas protegidas por PrivateRoute, roteadas por Role (cliente vai para /dashboard, funcionario vai para /admin).

## FASE 5 — Responsividade [Peso: 1,0 pt] (Sessão 7–8)

---

### Tarefa 5.1 — Media queries nos arquivos CSS existentes [45 min]

Adicione ao final de cada arquivo CSS. Ponto de quebra principal: 768px.



```
@media (max-width: 768px) {
  /* Auth.css */
  .inpt-form { width: 85vw; }
  .container-auth { padding: 20px; }

  /* LandingPage.css */
  .conteudo-landing { max-width: 90vw; margin-left: 5%; }
  .texto-landing { font-size: 22px; letter-spacing: -1px; }
  .btns-landing { flex-direction: column; }
  .btns-landing button { width: 100%; }
  .container-landing { background-position: center; }
}
```

### Tarefa 5.2 — Navbar responsiva com menu hambúrguer [30 min]

Adicione um botão hambúrguer (■) que aparece abaixo de 768px e toggle um menu vertical com os links de navegação. Implemente via useState em Navbar.jsx.

## FASE 6 — Deploy 24/7 [Peso: 0,75 pt] (Sessão 8–9)

- **Frontend → Vercel (gratuito):** Conecte o repositório GitHub ao Vercel. Defina VITE\_API\_URL como variável de ambiente apontando para a URL do backend.
- **Backend → Railway ou Render (gratuito):** Faça deploy do projeto .NET. Configure as variáveis de ambiente (ConnectionStrings\_\_DefaultConnection, Jwt\_\_Key) no painel do serviço — nunca no código.
- **Configure CORS no Program.cs:** Permita requests da origem do Vercel.

```
builder.Services.AddCors(options => options.AddPolicy("frontend",
  policy => policy.WithOrigins("https://seu-app.vercel.app")
    .AllowAnyHeader().AllowAnyMethod()));

// Depois do Build():
app.UseCors("frontend");
```

## FASE 7 — Documentação e Organização Final [Peso: 0,5 + 1,0 pt] (Sessão 9–10)

### Tarefa 7.1 — XML Doc Comments no C# [20 min]

```
/// <summary>
/// Autentica um cliente e retorna um token JWT.
/// </summary>
/// <param name='dto'>Credenciais de acesso (email e senha).</param>
/// <returns>Token JWT com validade de 8 horas.</returns>
[HttpPost("login")]
public async Task<IActionResult> Login([FromBody] LoginDto dto) { ... }
```

### Tarefa 7.2 — Atualizar README.md [15 min]

- Corrija a stack: remova MongoDB, documente PostgreSQL + Supabase.
- Adicione seção de instalação local (dotnet run, npm run dev).
- Adicione link do deploy em produção (Vercel + Railway URLs).
- Adicione diagrama simples de arquitetura (pode ser ASCII art).

### **Tarefa 7.3 — Limpar o repositório [10 min]**

- Remova bin/ e obj/ do git (adicione ao .gitignore).
- Remova RevisaoController.cs vazio ou preencha-o (Fase 4).
- Padronize o naming dos Models (todos usam Id, não IdFuncionario).

## 5. Resumo Visual — Ordem de Execução

| Fase  | O que fazer                                              | Pontos em jogo       | Estimativa   |
|-------|----------------------------------------------------------|----------------------|--------------|
| 1     | Fundação: fix router, instalar deps, remover credenciais | Base para tudo       | ~30 min      |
| 2     | Autenticação JWT + BCrypt + PrivateRoute no React        | 1,0 pt (crit. 1 e 2) | ~3 h         |
| 3     | PUT atualizar cadastro + Página de Perfil                | 0,5 pt (crit. 3)     | ~1 h         |
| 4     | RevisaoController + Kanban/Dashboard                     | 0,75 pt (crit. 5)    | ~2,5 h       |
| 5     | Responsividade CSS + Navbar mobile                       | 1,0 pt (crit. 6)     | ~1,5 h       |
| 6     | Deploy Vercel (frontend) + Railway (backend) + CORS      | 0,75 pt (crit. 7)    | ~1 h         |
| 7     | XML Docs, README correto, limpeza do repo                | 1,5 pt (crit. 8 e 9) | ~1 h         |
| TOTAL | —                                                        | ~5,5 pts recuperados | ~10–11 horas |

A **Fase 1** é obrigatória antes de qualquer outra coisa — sem o router corrigido, o frontend não funciona em absoluto. A **Fase 2** (autenticação) é o coração do projeto e a que mais pontos vale: não a deixe para o final. Com as Fases 1 a 4 completas, você já cobre 2,75 pts dos 3,5 pts em jogo nas funcionalidades. Responsividade e deploy são os mais rápidos de implementar versus o retorno em nota — priorize-os antes da documentação.